

5A)SAFETY ALGORITHM

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], finish[n], safe[n];

    // Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Maximum Matrix
    printf("Enter Maximum Demand Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Available Resources
    printf("Enter Available Resources:\n");
    for(j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
    }

    // Need Matrix = Max - Allocation
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
}
```

```

// Initialize finish as false
for(i = 0; i < n; i++) {
    finish[i] = 0;
}

int count = 0;

while(count < n) {
    int found = 0;

    for(i = 0; i < n; i++) {

        if(finish[i] == 0) {

            int possible = 1;

            // Check Need <= Available
            for(j = 0; j < m; j++) {
                if(need[i][j] > avail[j]) {
                    possible = 0;
                    break;
                }
            }

            // Process can execute
            if(possible) {

                for(k = 0; k < m; k++) {
                    avail[k] += alloc[i][k];
                }

                safe[count] = i;
                finish[i] = 1;
                count++;
                found = 1;
            }
        }
    }

    // Unsafe state
    if(found == 0)
        break;
}

```

```
// Check safe state
if(count == n) {
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");

    for(i = 0; i < n; i++) {
        printf("P%d", safe[i]);

        if(i != n - 1)
            printf(" -> ");
    }
}
else {
    printf("\nSystem is NOT in a safe state.\n");
}

return 0;
}
```

```
RIDHI GANESH x + v
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0) execution time : 49.122 s
Press any key to continue.
|
```

5B) DEADLOCK DETECTION

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m;
    int i, j;
```

```
    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```

scanf("%d", &m);

int alloc[n][m];
int req[n][m];
int avail[m];
int finish[n];
int safe[n];

// Allocation matrix
printf("Enter allocation matrix:\n");
for(i = 0; i < n; i++) {
    printf("Process %d: ", i);
    for(j = 0; j < m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}

// Request matrix
printf("Enter request matrix:\n");
for(i = 0; i < n; i++) {
    printf("Process %d: ", i);
    for(j = 0; j < m; j++) {
        scanf("%d", &req[i][j]);
    }
}

// Available resources
printf("Enter available resources: ");
for(i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}

// Initially all are unfinished
for(i = 0; i < n; i++) {
    finish[i] = 0;
}

int count = 0;

while(count < n) {

    int found = 0;

    for(i = 0; i < n; i++) {

```

```

if(finish[i] == 0) {

    int possible = 1;

    // Check request <= available
    for(j = 0; j < m; j++) {

        if(req[i][j] > avail[j]) {
            possible = 0;
        }
    }

    // If possible execute process
    if(possible == 1) {

        for(j = 0; j < m; j++) {
            avail[j] = avail[j] + alloc[i][j];
        }

        safe[count] = i;
        count++;
        finish[i] = 1;
        found = 1;
    }
}

// Unsafe state
if(found == 0) {
    break;
}

// Final Output
if(count == n) {

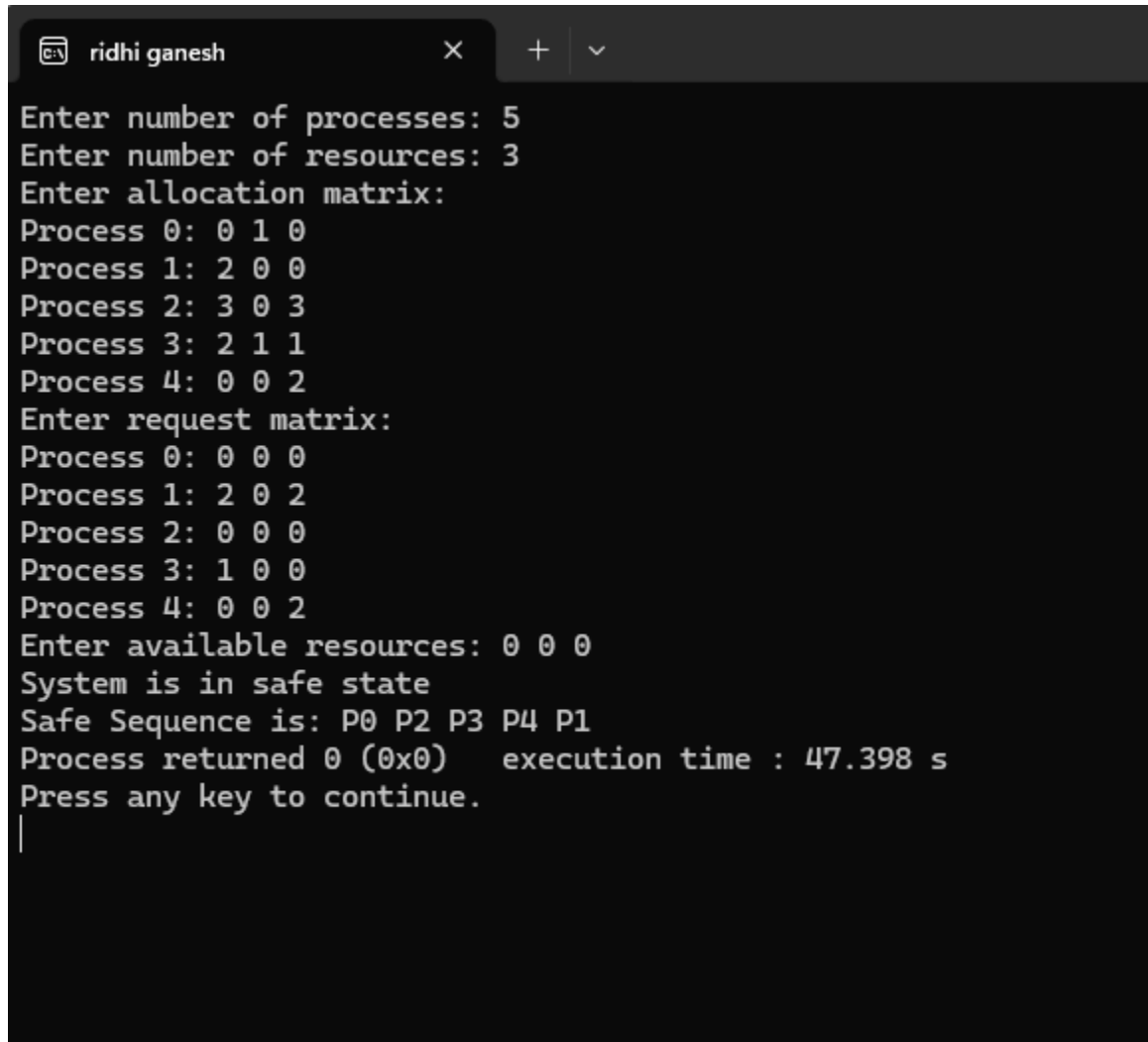
    printf("System is in safe state\n");

    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++) {
        printf("P%d ", safe[i]);
    }
}

```

```
}  
else {  
  
    printf("System is NOT in safe state");  
}  
  
return 0;  
}
```



```
ridhi ganesh X + v  
Enter number of processes: 5  
Enter number of resources: 3  
Enter allocation matrix:  
Process 0: 0 1 0  
Process 1: 2 0 0  
Process 2: 3 0 3  
Process 3: 2 1 1  
Process 4: 0 0 2  
Enter request matrix:  
Process 0: 0 0 0  
Process 1: 2 0 2  
Process 2: 0 0 0  
Process 3: 1 0 0  
Process 4: 0 0 2  
Enter available resources: 0 0 0  
System is in safe state  
Safe Sequence is: P0 P2 P3 P4 P1  
Process returned 0 (0x0)   execution time : 47.398 s  
Press any key to continue.  
|
```

1WA24CS229